

Proof Protocol: A Verifiable Execution Layer for AI-Generated and Programmatic Computation

Anonymous¹

¹Preprint — submitted to arXiv

June 27, 2026

Abstract

We present PROOF PROTOCOL, a system that transforms arbitrary computational claims — including those generated by large language models (LLMs) — into verifiable, reproducible, and cryptographically signed execution artifacts called *Computational Proof Objects* (CPOs). A CPO bundles a natural-language claim, executable code, a controlled sandbox environment, deterministic execution results, a SHA-256 content hash, and an Ed25519 digital signature issued by the executing node. Together, these fields form an immutable, auditable record of a computation.

Unlike zero-knowledge proof systems, PROOF PROTOCOL requires no trusted setup, no cryptographic circuit compilation, and no global consensus: verification consists of re-executing the same code under the same constraints and comparing hashes. We describe the architecture, formalize the CPO data model, analyze the security properties, and demonstrate that the approach is lightweight enough to deploy as a REST API service today. The system natively supports multiple isolated execution environments — which we call *worlds* — covering symbolic mathematics, probabilistic inference, evolutionary computation, formal verification, and multimodal data processing.

Contents

1	Introduction	3
2	The CPO Data Model	3
3	System Architecture	4
4	Execution Model	5
4.1	Sandbox Isolation	5
4.2	Bounded Determinism	5
5	Cryptographic Layer	5
5.1	Canonical Serialization	5
5.2	Content Hash	6
5.3	Node Identity and Signature	6
6	World Abstraction	6

7	Verification Procedure	6
8	Security Analysis	7
8.1	Threat Model	7
8.2	Non-Goals	7
9	Limitations	7
10	Related Work	8
11	Conclusion	8

1 Introduction

Modern AI systems and automated computation pipelines share a common weakness: their outputs are difficult to verify independently. A language model might produce a plausible-sounding numerical answer that is arithmetically incorrect; a data-science pipeline might produce different results on different machines; an autonomous agent might take actions whose causal chain is impossible to reconstruct after the fact.

Three distinct communities experience this problem independently:

AI practitioners observe that LLM outputs can *hallucinate* facts, including mathematical results, without any signal to the consumer that the output is unreliable [Ji et al., 2023].

Scientists struggle with *computational reproducibility*: a 2016 survey found that more than 70% of researchers had failed to reproduce another scientist’s results, with software environment differences as a major contributing factor [Baker, 2016].

Auditors and compliance engineers need *auditability*: the ability to prove *post hoc* that a particular output was produced by a particular computation running in a particular environment.

Existing solutions address each concern in isolation. Reproducibility tools (MLflow [MLflow, 2018], Weights & Biases) log hyperparameters but not execution semantics. Containerization (Docker, Nix) improves environment reproducibility but does not bind execution to its output cryptographically. Verifiable computation systems (Truebit [Deutsch & Reitwiessner, 2019], RISC Zero [RISC Zero, 2022]) provide strong guarantees via zero-knowledge proofs but impose circuit compilation overhead that is orders of magnitude larger than the computation itself.

PROOF PROTOCOL occupies a different point in this design space: a *practical, lightweight verifiable execution layer* that cryptographically binds computation inputs to outputs without zero-knowledge machinery or blockchain consensus. The key insight is that *reproducible isolation is a sufficient substitute for cryptographic universality* in many real-world settings.

The rest of this paper is organized as follows. Section 2 defines the CPO data model formally. Section 3 describes the system architecture. Section 4 details the execution model and isolation guarantees. Section 5 presents the cryptographic layer. Section 6 introduces the world abstraction. Section 7 specifies the verification procedure. Section 8 analyzes the security model. Section 9 acknowledges limitations. Section 10 surveys related work. Section 11 concludes.

2 The CPO Data Model

Definition 1 (Computational Proof Object). *A Computational Proof Object (CPO) is an immutable record*

$$\mathcal{P} = (id, world, claim, code, env, R, h, \sigma, node, t)$$

where:

- $id \in \{0, 1\}^{128}$ is a globally unique identifier (UUIDv4).
- $world \in \mathcal{W}$ selects an execution environment from a finite set of registered worlds (Section 6).
- $claim \in \Sigma^*$ is a natural-language description of the hypothesis being tested.
- $code \in \Sigma^*$ is deterministic, executable source code that operationalizes the claim.

- env is an execution specification (resource constraints, optional image override).
- $R = (stdout, stderr, exit_code, runtime_ms)$ is the observed execution result.
- $h = \text{SHA-256}(\text{canon}(\mathcal{P} \setminus \{h, \sigma\})) \in \{0, 1\}^{256}$ is the content hash over the canonical serialization of all other fields.
- $\sigma = \text{Sign}(sk_{node}, h)$ is an Ed25519 signature produced by the node's private key.
- $node \in \{0, 1\}^{256}$ is the public key of the signing node.
- $t \in \mathbb{Z}$ is the UTC timestamp of creation.

The hash h covers every field *except* h and σ themselves, so tampering with any single field invalidates both. Crucially, h is computed over a *canonical* serialization (Section 5) so that JSON key ordering or whitespace differences cannot produce false inequalities.

3 System Architecture

Figure 1 illustrates the end-to-end pipeline.

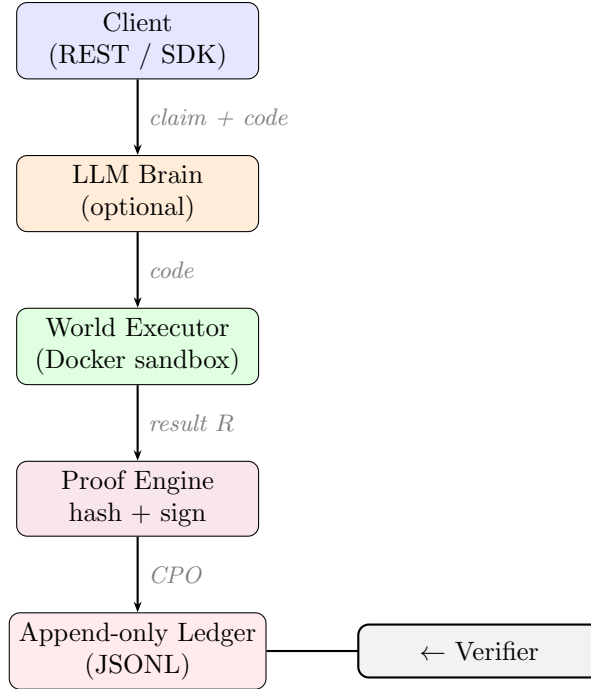


Figure 1: End-to-end pipeline of PROOF PROTOCOL. The LLM brain is optional; clients may submit code directly via `POST /prove`. The `POST /ask` endpoint activates the full pipeline including code generation. Any stored CPO can be retrieved and re-verified at any time.

The system exposes four primary REST endpoints:

- `POST /prove` — accept $(world, claim, code)$, execute, produce and store a CPO; return its hash and signature.

- `POST /ask` — accept a natural-language *question*, generate *code* via an LLM, then delegate to `/prove`.
- `GET /verify/{hash}` — retrieve a CPO by content hash, re-execute its code, and compare results.
- `GET /ledger` — browse the append-only ledger, optionally filtered by world.

4 Execution Model

4.1 Sandbox Isolation

Each execution is performed inside an ephemeral Docker container with the following constraints enforced at the container-runtime level:

- **Network isolation:** `network.disabled=True` prevents any outbound or inbound traffic.
- **Read-only filesystem:** `read_only=True` prevents persistent side effects.
- **Memory cap:** default 128 MiB, configurable per CPO via `execution_spec.constraints`.
- **Execution timeout:** default 10 s wall-clock time, also configurable.
- **Privilege suppression:** `no-new-privileges:true` prevents privilege escalation via `setuid` binaries.

These constraints together define what we call a *bounded execution environment*.

4.2 Bounded Determinism

We adopt a pragmatic definition of determinism:

Definition 2 (Bounded Determinism). *A computation $(code, env)$ is boundedly deterministic if two independent executions of code in environments that are structurally identical (same container image, same constraints, same input) produce identical observable outputs (stdout, exit_code).*

This is weaker than formal determinism: it does not account for non-deterministic floating-point hardware behavior across CPU generations, library version drift, or intentionally stochastic code. For the use cases targeted here — mathematical computation, symbolic reasoning, and deterministic algorithms — bounded determinism holds in practice.

5 Cryptographic Layer

5.1 Canonical Serialization

To ensure that h is stable across JSON implementations, we serialize the CPO (excluding h and σ) using an RFC 8785-inspired canonical form: keys are sorted lexicographically at every nesting level, no extra whitespace is inserted, and Unicode characters are preserved without escaping. Formally, for an object O :

```
canon(O) = JSON.dumps(O, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
```

5.2 Content Hash

$$h = \text{SHA-256}(\text{canon}(\mathcal{P} \setminus \{h, \sigma, \text{node}\}) \text{ encoded as UTF-8}) \quad (1)$$

5.3 Node Identity and Signature

Each node generates an Ed25519 keypair on first startup and persists the private key in a PEM file. The node’s identity is:

$$\text{node_id} = \text{SHA-256}(\text{pub_bytes})[:16] \text{ (hex prefix)}$$

The signature is:

$$\sigma = \text{Ed25519.Sign}(sk, h \cdot \text{UTF-8})$$

where h is the hex-encoded hash string of Equation (1).

Property 1 (Tamper Evidence). *For any CPO $\mathcal{P}$, modifying any field $f \notin \{h, \sigma\}$ produces $h' \neq h$, which causes $\text{Verify}(\text{node}, h', \sigma) = \perp$.*

6 World Abstraction

A *world* is a named, versioned Docker image that provides a specific runtime environment. Each world defines a domain of computation with its own library ecosystem and behavioral assumptions.

Table 1: Registered worlds and their runtime environments.

World	Primary domain	Key libraries	Verification mode
llm	General Python / LLM tools	stdlib	re-execution
symbolic	Symbolic mathematics	SymPy, Z3	re-execution
bayesian	Probabilistic inference	NumPyro, PyMC	bounded re-exec
evolutionary	Agent-based simulation	DEAP, Mesa	bounded re-exec
formal	Logic and SMT solving	Z3, py-aiger	re-execution
multimodal	Vision and audio	PyTorch CPU, Pillow	re-execution
neuro	Neural pipeline tooling	DSPy, LangChain	bounded re-exec

Worlds with *bounded re-execution* (stochastic by nature) can still be verified for signature validity and environment integrity, even if exact output reproduction is not guaranteed without fixing the random seed in the submitted code.

7 Verification Procedure

Given a content hash h , the verification procedure Verify is:

1. **Retrieve:** look up $\mathcal{P}$ in the ledger by h .
2. **Signature check:** $\text{Ed25519.Verify}(\mathcal{P}.\text{node}, h, \mathcal{P}.\sigma) \stackrel{?}{=} \top$
3. **Re-execute:** run $\mathcal{P}.\text{code}$ in world $\mathcal{P}.\text{world}$ with the same constraints to obtain fresh result R' .
4. **Output match:** $R'.\text{stdout.strip}() \stackrel{?}{=} \mathcal{P}.R.\text{stdout.strip}()$

5. **Exit code match:** $R'.exit_code \stackrel{?}{=} \mathcal{P}.R.exit_code$

The CPO is considered *verified* if and only if all five checks pass. Steps 1–2 require no re-execution and provide integrity and authenticity independently of determinism.

8 Security Analysis

8.1 Threat Model

We consider the following adversaries:

Passive ledger reader. An adversary who reads CPOs from the ledger but cannot modify them. The append-only structure and cryptographic hashing prevent undetected modification.

Malicious code submitter. An adversary who submits code intended to escape the sandbox or affect the host. Network isolation, read-only filesystem, seccomp profiles, and no-new-privileges collectively reduce the attack surface.

Compromised node. An adversary who controls the node’s private key. Signatures from a compromised key remain verifiable, but the node identity allows attribution. Key rotation invalidates future attestations but not past ones (since past CPOs carry the correct public key inline).

8.2 Non-Goals

PROOF PROTOCOL explicitly does not provide:

- **Semantic correctness:** a signed CPO proves that code *ran*, not that the code *correctly answers* the claim.
- **Global consensus:** there is no mechanism to prevent two nodes from producing conflicting CPOs for the same claim.
- **Kernel-level sandbox security:** Docker-based isolation does not protect against kernel exploits. Firecracker microVMs [Brubaker et al., 2020] or gVisor would strengthen this boundary at higher operational cost.
- **Confidentiality:** CPOs contain the executed code and its output in plaintext.

9 Limitations

Determinism scope. Stochastic worlds (Bayesian, Evolutionary) cannot guarantee identical outputs across re-executions without explicit random seed management by the code author.

Container security. Docker shares the host kernel. A kernel-level exploit may escape the sandbox. Production deployments should use microVM isolation.

LLM code quality. The `/ask` endpoint relies on an LLM to generate correct code. A hallucinated or incorrect code snippet produces a signed CPO for a wrong answer. The signature attests to *execution*, not to *correctness*.

Ledger scalability. The current JSONL ledger is append-only and single-file. At scale, this requires partitioning by world or time range, or migration to a structured event store.

No cross-node consensus. Each node is independently trusted. A network of nodes providing quorum-based verification is a planned extension, not a current guarantee.

10 Related Work

Reproducible research. MLflow [MLflow, 2018] and Weights & Biases record hyperparameters and artifacts but do not cryptographically bind execution environments to outputs. Nix [Nix, 2006] and Guix provide reproducible builds but are scoped to software delivery, not runtime execution.

Verifiable computation. Truebit [Deutsch & Reitwiessner, 2019] introduces interactive proofs for on-chain verification of off-chain computation. RISC Zero [RISC Zero, 2022] and StarkWare compile arbitrary programs to ZK circuits, providing unconditional soundness at the cost of circuit compilation time. Cartesi [Bolosky et al., 2021] runs Linux applications in a reproducible RISC-V machine to generate verifiable computation traces. PROOF PROTOCOL trades ZK soundness for simplicity: verification is re-execution rather than proof checking.

Trusted Execution Environments. Intel SGX and AMD SEV provide hardware-level attestation of code running inside enclaves [Costan & Devadas, 2016]. These offer stronger isolation but require specific hardware and attestation infrastructure. PROOF PROTOCOL is hardware-agnostic.

Secure sandboxing. Firecracker [Brubaker et al., 2020] provides lightweight microVM isolation with sub-125 ms boot times, used in AWS Lambda and Fargate. gVisor [Schuster et al., 2021] intercepts syscalls in user space. Both are compatible with PROOF PROTOCOL’s execution layer and represent a natural upgrade path from Docker.

LLM grounding. Tool-augmented language models [Schick et al., 2023, Yao et al., 2023] address hallucination by grounding LLM outputs in external tool calls. PROOF PROTOCOL’s `/ask` endpoint can be viewed as an instance of this paradigm where the tool is a verifiable code executor.

11 Conclusion

We have presented PROOF PROTOCOL, a lightweight verifiable execution layer that transforms computational claims into signed, auditable, and reproducible artifacts. The system is grounded in three principles:

1. **Isolation implies reproducibility:** bounded execution environments make re-verification possible without ZK machinery.
2. **Canonical hashing implies integrity:** any tampering with a CPO is detectable by recomputing h .
3. **Signed attestation implies non-repudiation:** node identity is cryptographically bound to every CPO it issues.

Together, these properties position PROOF PROTOCOL as a practical trust layer for AI pipelines, scientific computation, and autonomous agents.

Availability. A reference implementation in Python (FastAPI, Ed25519, Docker) is available at <https://github.com/haynbroit-alt/CP0>.

References

- Baker, M. (2016). 1,500 scientists lift the lid on reproducibility. *Nature*, 533(7604):452–454.
- Machado, D., et al. (2021). Cartesi: Bringing real-world computations to the blockchain. *arXiv preprint arXiv:2109.04316*.
- Agache, A., et al. (2020). Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX NSDI*, pp. 419–434.
- Costan, V., and Devadas, S. (2016). Intel SGX explained. *IACR Cryptology ePrint Archive*, 2016:086.
- Ji, Z., et al. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- Zaharia, M., et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45.
- Dolstra, E. (2006). *The Purely Functional Software Deployment Model*. PhD thesis, Utrecht University.
- Lacasse, N., et al. (2020). gVisor: An application kernel for containers. *USENIX HotCloud*.
- Teutsch, J., and Reitwießner, C. (2019). A scalable verification solution for blockchains. *arXiv preprint arXiv:1908.04756*.
- Schick, T., et al. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in NeurIPS 36*.
- Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. In *ICLR 2023*.
- RISC Zero. (2022). RISC Zero: A general-purpose zero-knowledge virtual machine. <https://www.risczero.com>.